EPFL - Computer Science

COM-402 Information Security and Privacy

Lecture 07 — November 3, 2025

Author: Jonathan Dutoit

Professor: Mathias Payer

Automated Testing

Learning goals for today

- Differentiate between testing approaches (manual, semi-automatic, automatic)
- Leverage fuzzing as a software testing technique
- Understand input generation and mutation to effectively explore targets
- Learn about sanitization methods to detect more bugs
- Identify opportunities for improving fuzzers
- Explain bug oracles and how they signal faults

Third-party library vulnerabilities can propagate to commercial software and be used for exploitation, e.g., in Chrome. A bug in a third-party component corrupts memory in the renderer process.

Why testing?

Testing is the process of analysing a program to find errors. An error is simply a deviation between observed behavior and specified behavior, i.e., a violation of the underlying specification:

- Functional requirements (features a, b, c)
- Operational requirements (performance, usability)
- What about security requirements?

Testing can only show the presence of bugs, never their absence!

> **(Edsger W. Dijkstra)**
>
> A successful test find a deviation. Testing is a form of program analysis. Code is analyzed, the testing environment observers the behavior of the code, detecting violations.

Turtles All the Way Down: Bugs are Everywhere

Giant code bases

- Chromium, Linux Kernel, GNOME, etc...
- Millions of LoC result in a huge attack surface
- Millions of vulnerable hosts, bug is weaponizable

Dependency inflation

- X depends-on A depends-on 1 depends-on ..., ...
- Redundant code, even bigger attack surface
- Single point-of-failure (e.g., Log4j)

System abstraction

- Developers unaware of underlying assumptions
- ChatGPT-generated code

Integration hell

- 2 "bug-free" systems can still interact in a faulty manner

Software Testing and Verification

Manual Testing: done manually by the developer

- Code reviews
- Unit and regression tests (JUnit, libcheck) are incomplete but sound, only code with tests is tested but any failed tests indicate an actual issue

Semi-automated Testing: combines manual efforts with automated tools

- Annotation (pre- and post-conditions) and proof engines (F* lang)
- Type checking and dependent types (Coq, F*)
- Code bug pattern search (CodeQL)

Automated Testing: relies on automated tools without a human-in-the-loop

- Static analysis is neither sound (only reports true bugs) nor complete (report all bugs) due to lack of runtime information but remains an indicator of possible bugs.
- Dynamic analysis (**fuzzing**, symbolic execution) is incomplete but sound due to state explosion but any report issues are true bugs.

Manual Testing

Code Reviews

Code review improves software quality through peer review, i.e., reviewers read (parts of) the code and specification to find flaws and to highlight code smells

- Flaw: any violation of the specification or bug

- Code smells: not per-se an issue but could become one (target to improve)

Goals: improve code quality, find flaws, learning/knowledge transfer, foster ideas/alternate solutions, complicate to standards (e.g., standards may require certain code reviews)

Code reviews can never find all flaws, but offer different thoroughness. Different types of code reviews: inspection, walk-through, or audit.

Types of Code Reviews: Inspection

During an **inspection**, a group of reviewers goes through all code line-by-line to find flaws and to document any unexpected behavior.

- Multiple developers thoroughly review code
- All interactions with dependencies are inspected
- Validates current snapshot of code base
- "Stop the world" form of code review, as no concurrent progress possible
- Most thorough form of code review (but also most expensive)
- Goal: sporadically check and review all code

Types of Code Reviews: Walk-through

In a **walk-through**, whenever a new feature is available, check that feature and how it interacts with the rest of the system.

- Code is reviewed incrementally, assumes the rest of the code remains correct
- Gradual reviews of code base, focusing on new code
- Review changes (i.e., pull requests or commits), before they are integrated
- Better suited to modern project development
- Pioneered by Google's code review practices
- Focus is on improving code quality
- Goal: continuously validate code as it is integrated

Types of Code Reviews: Audit

During an **audit**, a (often external) group of reviewers spot-checks the code to find flaws, often searching patterns to find flaws/violations

- Not all code is checked, but spot checks identify issues
- Assign fixed amount of time, assume incompleteness
- Audits focus on key issues, not fine details
- Goal: get an overview of code quality and identify main flaws

Unit and Regression Tests

A **unit test**, executes and tests a fragment of code (e.g., function or file).

Developers write unit tests alongside the functional code to quickly test it during development. There are *positive* tests to ensure code accept and correctly processes valid input and *negative* tests to ensure code rejects invalid inputs.

Whenever code is changed, unit tests are run to protect against regression.

When finding a bug, add a unit test to trigger the bug and test that the bug is correctly fixed to protect against **regression**.

Semi-Automated Testing

Static Analysis: Compile Time Warnings

Goal: highlight issues in code, give feedback to developer

Must be fast (low overhead, simple analysis) and must have low false positives (warning fatigue).

`clang -Wall -Wextra -Wpedantic`

- Wall: enables basic set of warnings, mostly easy to fix
- Wextra: extends Wall, being a little more detailed; examples are empty function bodies, unused parameters, or sign mismatches
- Wpedantic: warns about anything that goes beyond strict C/C++, highlights areas of bad code style as well
- Weverything: turn on every single warning that exists; creates noise

Static Analysis

A static analysis reasons about the code of executing it.

The line between static and dynamic analysis is blurry. Generally: a static analysis looks only at the code without keeping track of concrete values during execution.

Advantage of static analysis:

- Full coverage (no need for complete test cases)
- Scales to large code bases (but test cases may miss edge cases)
- Abstract interpretation (no need for a runtime environment)

Disadvantage (having a lot of false positives):

- Computation depends on data, resulting in undecidability
- Over-approximation due to imprecision and aliasing leading to false positives

Annotations

Annotations augment source doe but are not part of the functional code itself. Compilers may use annotations to test or check certain properties. For example:

- Types in python
- Argument usage in C/C++
- Pre/postconditions for verification

CodeQL: Analysis Framework to Find Bugs

CodeQL is a pattern-based code analysis engine to automate security checks. Target code is parsed and analyzed based on patterns in the query language, starting from sources the patterns search for a path towards a sink.

Analysis is performed on a database that stores all the **compilation information**

Quality of the reported bugs depends on the quality of the queries (heuristics)

SemGrep: Find bugs variants with source-code patterns

SemGrep is a lightweight static analysis tool for bug discovery. It's similar to CodeQL, finds vulnerable code that match an input pattern.

It **does not require compilation**. On the other hand, it cannot reason about **variable types**.

The 8 "Laws" of Static Analysis

- Can't check code you don't see: must fit into build system
- Can't check code you can't parse: must parse "real" code
- Not everything is implemented in C: must handle other languages
- Not a bug: developers will not fix any report
- Not all bugs matter: developers must prioritize
- False positives matter: reduce developer burden
- False negatives matters: reduce developer burden
- Annotations are extremely costly: reduce developer burden

Automated Testing

Formal Verification: Model Checking / Symbolic Execution

Instead of running the program, translate it into a formula, and ask constraint solver about violation on all feasible/possible paths

Formal Verification

Formal verification is the act of ***proving*** or disproving the ***correctness of a system*** with respect to a certain formal specification or property

- Model checking: translate code into state machine, model checker analyzes state machine
- Symbolic execution: engine analyzes program by exploring paths

(Bounded) Model Checking

The analyst translates the code and specification into a finite state machine. The model checker then verifies that no illegal states are reachable.

While this allows end-to-end verification, it is highly labor intensive.

Bounded model checking slices the program end checks the slices instead to scale a bit further (1000s LoC instead of 100s LoC)

It's useful for small software: pacemeaker, etc... **It doesn't scale to large software!**

Symbolic Execution

Values treated as symbols without concrete values.

Define set of conditions at code locations

- Symbolic Execution determines triggering input

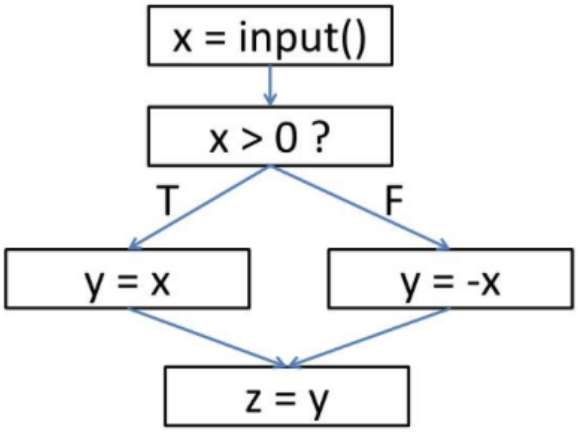
Testing: finding bugs in applications

- Infer pre/post conditions and add assertions
- Use symbolic execution to negate conditions

Generating PoC input through SAT/SMT solving

```
x = input()
if(x > 0)
  y = x;
else
  y = -x;

z = y;
```



Program verification: z = |x|

z = ite(x>0, x, -x) → SMT solver → Valid?

Symbolic Execution: Challenges

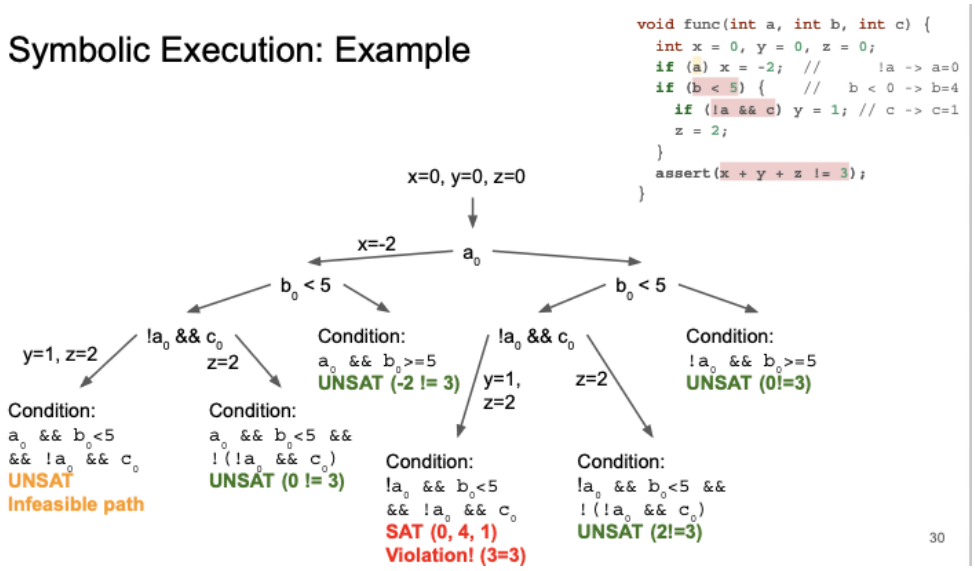
Bug conditions must be clearly defined.

State explosion: either the length of the constraints or the amount of state doubles at each loop iteration/conditional

- Searching valid paths (depth/breadth first) is crucial
- Path optimization key research are
- State pruning another key are

Complexity limits scalability.

> Symbolic execution is complete in theory (when all paths are checked) but incomplete in practice (due to state explosion)



Concolic Execution

Symbolic execution is limited due to state exposition (too many paths to evaluate). One way to improve is to limit the exploration close to a real path.

Concolic execution starts from a real execution. abstracts it, and searches nearby.

Path is hard-coded but remaining values are explored for violations.

Fuzzing

Automatic and dynamic software testing technique.

Key concept: fastest way to test a program is to run it.

Analogy: obstacle course

- Programs are riddle with control-flow decisions

- Only one path can be taken at a time
- `path[-1]` may depend on `path[0:-1]`

Fuzzing Obstacles

Fuzzers are hindered by control-flow and data-flow restrictions

- **Magic byte matching**
- **Checksum**
- **Compression/Encryption**
- **Dependencies**
- **Indirection**

Fuzzer Best Practices

A fuzzer should *explore* the program

- A bug cannot be found if its code is not executed

A fuzzer should *trigger* bugs

- Executing buggy code does not necessarily mean a bug has been triggered

A fuzzer should *identify* faults

- A triggered fault that goes unidentified is wasted effort

A fuzzer should be *fast*

- Speed is key to a fuzzer's efficiency
- Higher throughput = more inputs tested = higher change of triggering a bug if it exists

Types of Fuzzing

Blackbox Fuzzing: Doesn't reach deep into the program. Random input generation, no feedback, no learning, move fast, break things.

Greybox Fuzzing: Feedback guidance.

Most common feedback: coverage profiles

- Coverage-guided fuzzing
- Objective function: code coverage
- Metric: a execution counts of functions/lines/basic blocks/edges

Remains fast:

- Instrumentation is usually lightweight
- Sacrifices some CPU cycles for performance benefits

Fuzzing: Input Generation

Not all inputs are made equal.

Mutation-based generation doesn't required specification and inputs are seen as binary blocks. Mutators transform input (bit flips, int operations). Its fast, portable, scalable. Coverage feedback an objective function for a genetic algorithm.

Grammar-based generation does require specification for input structure and inputs are seen as parametric data. Parameters are sample, constrains are resolved to generate grammatically-valid inputs. May be slow, application-specific. It may be of interest to test exception-handling code in a program

Sanitization: Fault Detection

Fuzzers explore the program, possibly triggering bugs in the process and unidentified bugs are useless.

Sanitizers are fault detectors and reporters

- Verify security policies at runtime
- Detect many violations by may be imprecise (i.e., some violations may be missed)

Sanitization

Ideal sanitizer: detect bugs as soon as they occur. First approximation: CPU exceptions (segfault, timeouts, div by 0, int overflows)

LLVM sanitizers by policy:

- **Spatial and temporal memory safety**
- **Language-specific behavior**
- **Concurrency and ownership**

LLVM sanitizers can be applied to source code, at compile time.

Code is instrumented with checks at every possible location, e.g., just before a memory access.

Sanitization requires a model/abstraction/policy for expected behavior

- How to model logic or semantic bugs? Need specification
- Most specification is human-readable but not machine readable
- Can ChatGPT help? Probably, too slow for fuzzing

Identifying Faults

Crashes are a certain indicator of faults, but not all bugs crash.

Sanitizers implement fault detection by enforcing security policies.

Typical classes of bugs have existing sanitizers (e.g., memory corruption)

Real-world systems are complex and security-critical bugs can manifest as non-corrupting logic bugs, or bugs in integration

There is room for AI-guided bug detection, but it needs to be done carefully.

Comparison of Analysis

Technique	Scale (LoC)	False Negatives	False Positives	State Explosion
Formal Verification	100	None	No	High
Bounded Model Checking	1,000	Low	No	High
Symbolic Execution	10,000	None	No	High
Concolic Execution	50,000	Medium	No	Medium
Fuzzing	1,000,000	Medium	No	Low
Compiler Warning	100,000,000	High	Yes	No